

Likelihood to loss in practice

MAP, regularization and robust observation models, then output heads and stable NLLs.

THE QUESTION

Which noise model and which prior should shape the loss, and how do you implement it safely?

THE INTUITION

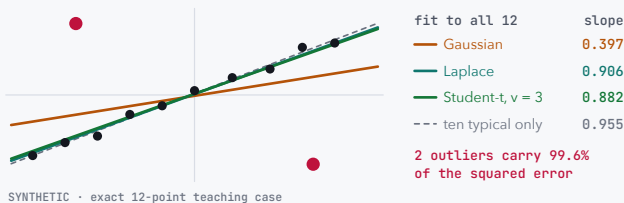
The noise model sets how hard each residual pulls: Gaussian noise lets two outliers flatten a line; Laplace and Student- t expect big errors. A prior adds $-\log p(\theta)$ to the loss: the regularizer.

KEY TAKEAWAY

Loss = $-\log \text{likelihood} - \log \text{prior}$. Choose both on purpose, use a stable NLL, and check one example by hand.

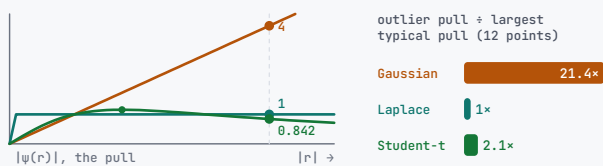
target semantics → noise model → prior on θ → parameterization → stable NLL + penalty → check by hand

1 Two outliers, three noise models



Ten points follow one trend; two sit far from it. Squaring lets those two dominate and flattens the least-squares slope. Laplace and Student- t keep all twelve yet track the trend: at $r = 4$ their densities are $\approx 68 \times$ the Gaussian's.

2 Robust regression is a modelling choice

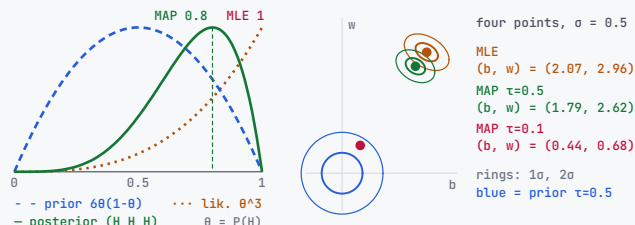


A point's pull is $\psi(r) = \partial \ell / \partial r$, with $r = y - \mu$:

Gaussian $\psi \propto r$; influence grows with $|r|$
Laplace $\psi \propto \text{sign}(r)$; constant magnitude
Student- t influence eventually falls for very large $|r|$

Inspect residuals. A heavy tail suits data where unusual observations are plausible; it is no substitute for fixing corrupt data.

3 A prior turns MLE into MAP



After H H H the MLE of $P(H)$ is 1; a prior favouring fair coins moves the peak to 0.8. MAP minimizes $-\log p(\mathcal{D}|\theta) - \log p(\theta)$. A Gaussian prior $\mathcal{N}(0, \tau^2 \mathbf{I})$ adds $\lambda \|\theta\|_2^2$ with $\lambda = 1/(2\tau^2)$; a Laplace prior adds L_1 . On the line, $\tau = 0.5$ ($\lambda = 2$) shrinks the MLE toward 0; $\tau = 0.1$ ($\lambda = 50$) underfits.

4 A practical model-selection map

target	distribution	head / NLL
real value	Gaussian	μ (and $\sigma > 0$) / MSE or Gaussian NLL
robust real	Laplace	μ (and $b > 0$) / absolute-error NLL
heavy-tail real	Student- t	$\mu, s > 0, \nu > 0$ / Student- t NLL
binary label	Bernoulli	one logit / BCE with logits
one of K count	Categorical Poisson	K logits / cross-entropy log-rate / Poisson NLL

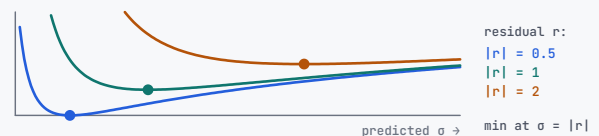
A loss can be optimized even when its likelihood story is inappropriate. The story tells you whether its behaviour matches the data.

5 Use logits, not rounded probabilities



Stable binary loss for logit z : $\ell(z, y) = \max(z, 0) - zy + \log(1 + e^{-|z|})$. Stable multiclass loss: $\ell(z, c) = -z_c + \text{logsumexp}(z)$. These fused forms never compute $\log(0)$ after sigmoid or softmax saturates.

6 Heteroscedastic regression



If uncertainty changes with the input, predict μ_i and $\sigma_i > 0$:

$$\ell_i = \frac{1}{2} \left[\frac{(y_i - \mu_i)^2}{\sigma_i^2} + \log \sigma_i^2 \right] + C.$$

The first term standardizes the residual; the second stops the model from making every σ_i arbitrarily large. **Validate** interval coverage and sharpness, not just point-error MSE.

The output activation belongs to the probabilistic model: it turns raw outputs into distribution parameters.

Debug a likelihood-based training pipeline

Check the mathematics, then the tensors, then the numerics, in that order. A decreasing loss alone is not enough.

1 · target, support → 2 · family → 3 · one example → 4 · reduction → 5 · stability → 6 · validation

7 First prove one example by hand

Pick one tiny prediction-target pair and compute the expected NLL:

binary $y = 1, p = 0.8$ $-\log 0.8 \approx 0.223$
 3-class $c = 2, p_c = 0.7$ $-\log 0.7 \approx 0.357$
 Gaussian $r = 2, \sigma = 1$ $r^2 / (2\sigma^2) = 2$ plus constants

Match the library's **unreduced** output to the hand value before training a model.

8 Shape audit

- ✓ Batch axis and class / event axes are named.
- ✓ Broadcasting is intentional, not accidental.
- ✓ A categorical target is an index, not a length- K float, unless soft labels are intended.
- ✓ Per-pixel or per-token masks remove padded observations before reduction.
- ✓ Start with `reduction="none"` and inspect every contribution.

9 Reduction and weighting

short, hard 2 2 mean over tokens = 0.88
 long, easy 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 mean of per-example = 1.25

illustrative token losses; the long example dominates a per-token mean

Decide what one training example contributes: $\mathcal{L} = \frac{1}{n} \sum_i w_i \ell_i$, $w_i \geq 0$. If an example has many pixels or tokens, choose whether to average within each example or across all observations: the two weight long examples differently.

Log the numerator and the denominator separately when using masks, class weights or variable-length sequences.

10 Numerical checks

- ✓ Logits, NLL and gradients are finite on the first batch.
- ✓ Positive scales are bounded away from zero.
- ✓ No duplicated sigmoid / softmax before a fused loss.
- ✓ Extreme logits are tested, not only values near zero.

11 Pause and think

Why not delete the outliers? Only if they are corrupt. If they are plausible, a heavy-tailed likelihood keeps them but bounds their pull.

How to pick ν , s or τ ? Compare held-out NLL on the same targets, or use domain knowledge.

12 Behavioural sanity tests

test	expected result
perfect direction	loss falls as the true-class logit rises or the residual shrinks
label permutation	performance collapses toward chance
constant baseline	trained model beats the mean / class-frequency predictor
duplicate batch	mean loss unchanged; summed loss doubles
one outlier	Gaussian reacts more strongly than Laplace / Student- t
zero probability	NLL tends to $+\infty$ for the observed event

13 Diagnose with the right residual

Regression: plot residuals against predictions and inputs; check spread and tails.

Classification: compare confidence with empirical accuracy; inspect per-class NLL.

Counts: compare the variance with the mean; strong overdispersion can invalidate Poisson.

A well-optimized misspecified likelihood can still give poor uncertainty and misleading gradients.

14 Library semantics to verify

<code>BCEWithLogitsLoss</code>	raw logits + float targets in $[0, 1]$
<code>CrossEntropyLoss</code>	raw logits + integer class indices; no prior softmax
<code>GaussianNLLLoss</code>	mean + variance, not standard deviation
<code>PoissonNLLLoss</code>	is the input a log-rate? are constants included?
<code>Distribution.log_prob</code>	inspect event and batch shapes before reducing

Names describe formulas, not tensor conventions. Read the API contract and test a hand-computed example.

15 Parameterize constraints safely

Let the network emit unconstrained raw values:

$p \in (0, 1)$ $p = \sigma(z)$
 $\mathbf{p} \in \Delta^{K-1}$ $\mathbf{p} = \text{softmax}(\mathbf{z})$
 $\sigma, \lambda, s > 0$ $\text{softplus}(a) + \varepsilon$, or model a log-scale
 $\nu > 0$ $\nu = \text{softplus}(u) + \varepsilon$; often a safer lower bound

Do not clamp learned probabilities as the primary model. Use a transformation whose range already obeys the constraints.

If the hand value, unreduced tensor and library disagree, fix the objective first; training cannot rescue it.

Robust and prior numbers from the Lecture 3 storyboard data (a synthetic 12-point case; H H H; the four points from $y = 2 + 3x$). The float32 loss curve and the σ curves are computed from their formulas. Slides, notebooks and summary videos: nipunbatra.github.io/dl-2026.